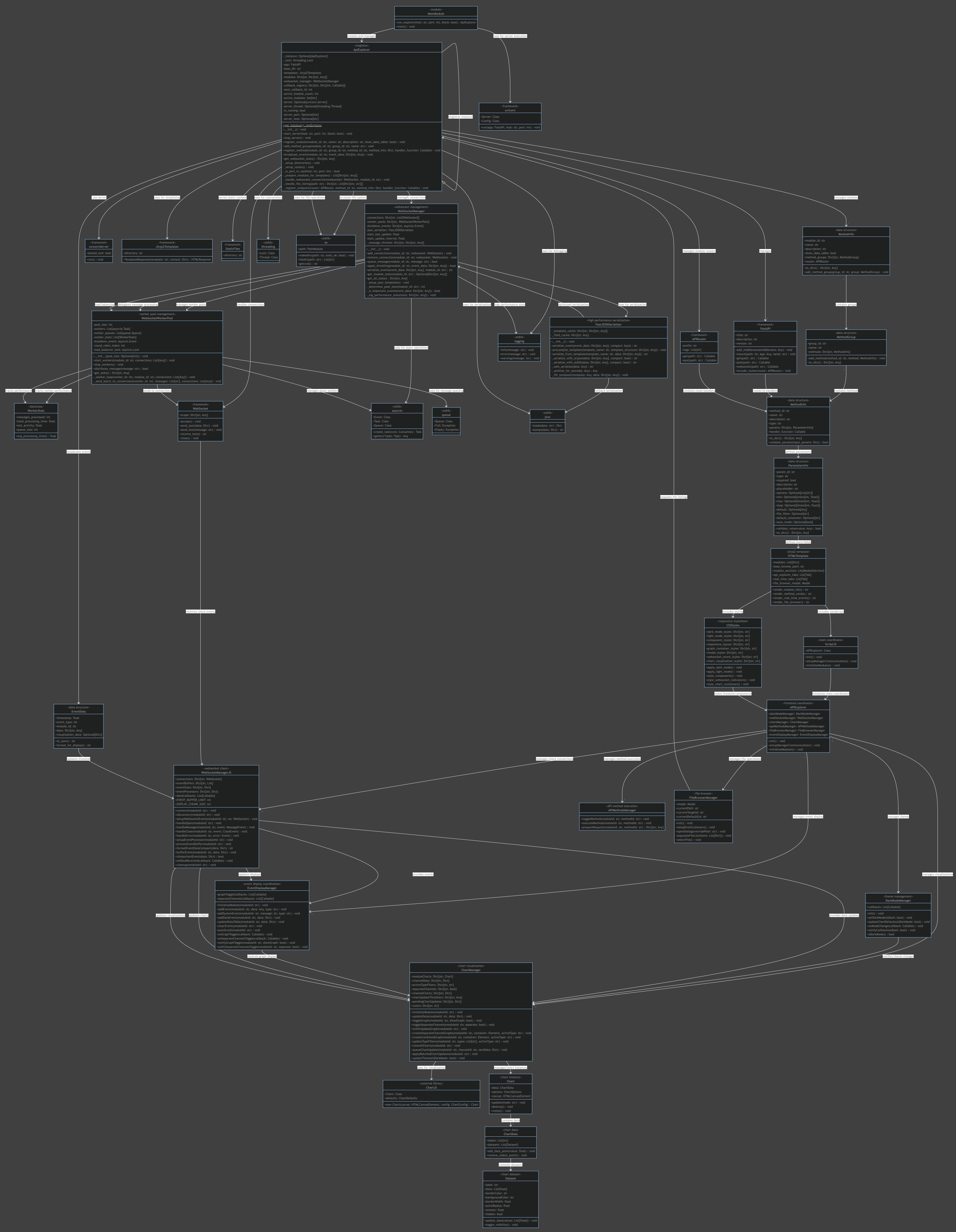
«module»
MainModule
+run_explorer(host: str, port: int, block: bool) : ApiExplorer
+main() : void
creates and manages
uses for server execution
«singleton»
ApiExplorer
-_instance: Optional[ApiExplorer]
-_lock: threading.Lock
-app: FastAPI
-base_dir: str
-templates: Jinja2Templates
-modules: Dict[str, Dict[str, Any]]
-websocket_manager: WebSocketManager
-callback_registry: Dict[str, Dict[int, Callable]]
-next_callback_id: int
-active_module_count: int
-active_modules: Set[str]
-server: Optional[uvicorn.Server]
-server_thread: Optional[threading.Thread]
-is_running: bool
-server_port: Optional[int]
-server_host: Optional[str]
+get_instance() : ApiExplorer
+__init__() : void
+start_server(host: str, port: int, block: bool) : void
+stop_server() : void
+register_module(module_id: str, name: str, description: str, show_data_table: bool) : void
+add_method_group(module_id: str, group_id: str, name: str) : void
+register_method(module_id: str, group_id: str, method_id: str, method_info: Dict, handler_function: Callable) : void
+broadcast_event(module_id: str, event_data: Dict[str, Any]) : void
+get_websocket_stats() : Dict[str, Any]
-_setup_directories() : void
-_setup_routes() : void
-_is_port_in_use(host: str, port: int) : bool
-_prepare_modules_for_template() : List[Dict[str, Any]]
-_handle_websocket_connection(websocket: WebSocket, module_id: str) : void
-_handle_file_listing(path: str) : Dict[str, List[Dict[str, str]]]
-_register_endpoint(router: APIRouter, method_id: str, method_info: Dict, handler_function: Callable) : void
singleton instance
«framework»
uvicorn
+Server: Class
+Config: Class
+run(app: FastAPI, host: str, port: int) : void
uns server
uses for templating
serves static content
uses for concurrency
uses for file operations
accesses file system
manages connections
manages modules
«framework»
uvicornServer
+should_exit: bool
+run() : void
«framework»
Jinja2Templates
+directory: str
+TemplateResponse(template: str, context: Dict) : HTMLResponse
«framework»
StaticFiles
+directory: str
«stdlib»
threading
+Lock: Class
+Thread: Class
«stdlib»
os
+path: PathModule
+makedirs(path: str, exist_ok: bool) : void
+listdir(path: str) : List[str]
+getcwd() : str
«websocket management»
WebSocketManager
-connections: Dict[str, List[WebSocket]]
-worker_pools: Dict[str, WebSocketWorkerPool]
-shutdown_events: Dict[str, asyncio.Event]
-json_serializer: FastJSONSerializer
-stats_last_update: float
-stats_update_interval: float
-_message_throttle: Dict[str, Dict[str, Any]]
+__init__() : void
+add_connection(module_id: str, websocket: WebSocket) : void
+remove_connection(module_id: str, websocket: WebSocket) : void
+queue_message(module_id: str, message: str) : bool
+apply_throttling(module_id: str, event_data: Dict[str, Any]) : bool
+serialize_event(event_data: Dict[str, Any], module_id: str) : str
+get_module_stats(module_id: str) : Optional[Dict[str, Any]]
+get_all_stats() : Dict[str, Any]
-_setup_json_templates() : void
-_determine_pool_size(module_id: str) : int
-_is_important_event(event_data: Dict[str, Any]) : bool
-_log_performance_stats(stats: Dict[str, Any]) : void
uses for debugging
manages module routers
manages
«data structure»
ModuleInfo
+module_id: str
+name: str
+description: str
+show_data_table: bool
+method_groups: Dict[str, MethodGroup]
+router: APIRouter
+to_dict() : Dict[str, Any]
+add_method_group(group_id: str, group: MethodGroup) : void
contains groups
load balancing
delegates message processing
manages worker pools
handles connections
uses for serialization
logs performance stats
optimized serialization
uses for serialization
«worker pool management»
WebSocketWorkerPool
-pool_size: int
-workers: List[asyncio.Task]
-worker_queues: List[queue.Queue]
-worker_stats: List[WorkerStats]
-shutdown_event: asyncio.Event
-round_robin_index: int
-load_balancer_lock: asyncio.Lock
+__init__(pool_size: Optional[int]) : void
+start_workers(module_id: str, connections: List[Any]) : void
+stop_workers() : void
+distribute_message(message: str) : bool
+get_stats() : Dict[str, Any]
-_worker_loop(worker_id: int, module_id: str, connections: List[Any]) : void
-_send_batch_to_connections(worker_id: int, messages: List[str], connections: List[Any]) : void
«stdlib»
logging
+info(message: str) : void
+error(message: str) : void
+warning(message: str) : void
«high-performance serialization»
FastJSONSerializer
-_template_cache: Dict[str, Dict[str, Any]]
-_field_cache: Dict[str, Any]
+__init__() : void
+serialize_event(event_data: Dict[str, Any], compact: bool) : str
+precompile_template(template_name: str, template_structure: Dict[str, Any]) : void
+serialize_from_template(template_name: str, data: Dict[str, Any]) : str
-_serialize_with_orjson(data: Dict[str, Any], compact: bool) : str
-_serialize_with_stdlib(data: Dict[str, Any], compact: bool) : str
-_safe_serialize(data: Any) : str
-_sanitize_for_json(obj: Any) : Any
-_fill_template(template: Any, data: Dict[str, Any]) : void
«framework»
APIRouter
+prefix: str
+tags: List[str]
+get(path: str) : Callable
+post(path: str) : Callable
«framework»
FastAPI
+title: str
+description: str
+version: str
+add_middleware(middleware: Any) : void
+mount(path: str, app: Any, name: str) : void
+get(path: str) : Callable
+post(path: str) : Callable
+websocket(path: str) : Callable
+include_router(router: APIRouter) : void
«data structure»
MethodGroup
+group_id: str
+name: str
+methods: Dict[str, MethodInfo]
+add_method(method_id: str, method: MethodInfo) : void
+to_dict() : Dict[str, Any]
uses for async operations
tracks performance
tracks worker performance
sends to connections
manages async workers
uses for message queuing
fallback serialization
contains route handlers
routes to handlers
contains methods
«dataclass»
WorkerStats
+messages_processed: int
+total_processing_time: float
+last_activity: float
+queue_size: int
+avg_processing_time() : float
«framework»
WebSocket
+scope: Dict[str, Any]
+accept() : void
+send_json(data: Dict) : void
+send_text(message: str) : void
+receive_text() : str
+close() : void
«stdlib»
asyncio
+Event: Class
+Task: Class
+Queue: Class
+create_task(coro: Coroutine) : Task
+gather(*tasks: Task) : Any
«stdlib»
queue
+Queue: Class
+Full: Exception
+Empty: Exception
«stdlib»
json
+loads(data: str) : Dict
+dumps(data: Dict) : str
«data structure»
MethodInfo
+method_id: str
+name: str
+description: str
+type: str
+params: Dict[str, ParameterInfo]
+handler_function: Callable
+to_dict() : Dict[str, Any]
+validate_params(input_params: Dict) : bool
broadcasts events
defines parameters
«data structure»
ParameterInfo
+param_id: str
+type: str
+required: bool
+description: str
+placeholder: str
+options: Optional[List[str]]
+min: Optional[Union[int, float]]
+max: Optional[Union[int, float]]
+step: Optional[Union[int, float]]
+default: Optional[Any]
+file_filter: Optional[str]
+default_extension: Optional[str]
+save_mode: Optional[bool]
+validate_value(value: Any) : bool
+to_dict() : Dict[str, Any]
requests file listings
defines form fields
«jinja2 template»
HTMLTemplate
+modules: List[Dict]
+base_browse_path: str
+module_sections: List[ModuleSection]
+api_explorer_tabs: List[Tab]
+real_time_tabs: List[Tab]
+file_browser_modal: Modal
+render_module_list() : str
+render_method_cards() : str
+render_real_time_events() : str
+render_file_browser() : str
receives client events
includes styling
includes JavaScript
«responsive stylesheet»
CSSStyles
+dark_mode_styles: Dict[str, str]
+light_mode_styles: Dict[str, str]
+component_styles: Dict[str, str]
+responsive_layout: Dict[str, str]
+graph_container_styles: Dict[str, str]
+modal_styles: Dict[str, str]
+websocket_event_styles: Dict[str, str]
+chart_visualization_styles: Dict[str, str]
+apply_dark_mode() : void
+apply_light_mode() : void
+style_components() : void
+style_websocket_indicators() : void
+style_chart_containers() : void
«main coordinator»
ScriptJS
+APIExplorer: Class
+init() : void
+setupManagerCommunication() : void
+initializeModules() : void
styles frontend components
initializes main coordinator
«data structure»
EventData
+timestamp: float
+event_type: str
+module_id: str
+data: Dict[str, Any]
+visualization_data: Optional[Dict]
+to_json() : str
+format_for_display() : str
«frontend coordinator»
APIExplorer
+darkModeManager: DarkModeManager
+webSocketManager: WebSocketManager
+chartManager: ChartManager
+apiMethodsManager: APIMethodsManager
+fileBrowserManager: FileBrowserManager
+eventDisplayManager: EventDisplayManager
+init() : void
+setupManagerCommunication() : void
+initializeModules() : void
updates frontend
manages client connections
manages method execution
manages file operations
«websocket client»
WebSocketManagerJS
+connections: Dict[str, WebSocket]
+eventBuffers: Dict[str, List]
+eventStats: Dict[str, Dict]
+eventProcessors: Dict[str, Dict]
+dataCallbacks: List[Callable]
+EVENT_BUFFER_LIMIT: int
+DISPLAY_CHUNK_SIZE: int
+connect(moduleId: str) : void
+disconnect(moduleId: str) : void
+setupWebSocketEvents(moduleId: str, ws: WebSocket) : void
+handleOpen(moduleId: str) : void
+handleMessage(moduleId: str, event: MessageEvent) : void
+handleClose(moduleId: str, event: CloseEvent) : void
+handleError(moduleId: str, error: Event) : void
+setupEventProcessor(moduleId: str) : void
+processEventBuffer(moduleId: str) : void
+formatEventDataCompact(data: Dict) : str
+bufferEvent(moduleId: str, data: Dict) : void
+isImportantEvent(data: Dict) : bool
+onDataReceived(callback: Callable) : void
+cleanup(moduleId: str) : void
«API method execution»
APIMethodsManager
+toggleMethod(moduleId: str, methodId: str) : void
+executeMethod(moduleId: str, methodId: str) : void
+prepareRequest(moduleId: str, methodId: str) : Dict[str, Any]
«file browser»
FileBrowserManager
+modal: Modal
+currentPath: str
+currentTargetId: str
+currentDefaultExt: str
+init() : void
+setupEventListeners() : void
+openDialog(overridePath: str) : void
+populateFileList(items: List[Dict]) : void
+selectFile() : void
manages event display
manages theme
manages visualizations
updates displays
provides events
«event display coordination»
EventDisplayManager
+graphToggleCallbacks: List[Callable]
+separateChannelsCallbacks: List[Callable]
+initializeModule(moduleId: str) : void
+addEvent(moduleId: str, data: Any, type: str) : void
+addSystemEvent(moduleId: str, message: str, type: str) : void
+addDataEvent(moduleId: str, data: Dict) : void
+updateDataTable(moduleId: str, data: Dict) : void
+clearEvents(moduleId: str) : void
+autoScroll(moduleId: str) : void
+onGraphToggle(callback: Callable) : void
+onSeparateChannelsToggle(callback: Callable) : void
+notifyGraphToggle(moduleId: str, showGraph: bool) : void
+notifySeparateChannelsToggle(moduleId: str, separate: bool) : void
updates visualizations
updates charts
provides data updates
«theme management»
DarkModeManager
+callbacks: List[Callable]
+init() : void
+setDarkMode(isDark: bool) : void
+updateChartDefaults(isDarkMode: bool) : void
+onModeChange(callback: Callable) : void
+notifyCallbacks(isDark: bool) : void
+isDarkMode() : bool
controls graph display
notifies theme changes
«chart visualization»
ChartManager
+moduleCharts: Dict[str, Chart]
+channelData: Dict[str, Dict]
+activeTypeFilters: Dict[str, str]
+separateChannels: Dict[str, bool]
+channelCharts: Dict[str, Dict]
+chartUpdateThrottlers: Dict[str, Any]
+pendingChartUpdates: Dict[str, Dict]
+colors: Dict[str, str]
+initializeModule(moduleId: str) : void
+updateData(moduleId: str, data: Dict) : void
+toggleGraph(moduleId: str, showGraph: bool) : void
+toggleSeparateChannels(moduleId: str, separate: bool) : void
+initOrUpdateGraph(moduleId: str) : void
+createSeparateChannelGraphs(moduleId: str, container: Element, activeType: str) : void
+createCombinedGraph(moduleId: str, container: Element, activeType: str) : void
+updateTypeFilters(moduleId: str, types: List[str], activeType: str) : void
+clearAllCharts(moduleId: str) : void
+queueChartUpdate(moduleId: str, channelId: str, newData: Dict) : void
+applyBatchedChartUpdates(moduleId: str) : void
+updateTheme(isDarkMode: bool) : void
uses for visualization
manages chart instances
«external library»
ChartJS
+Chart: Class
+defaults: ChartDefaults
+new Chart(canvas: HTMLCanvasElement, config: ChartConfig) : Chart
«chart instance»
Chart
+data: ChartData
+options: ChartOptions
+canvas: HTMLCanvasElement
+update(mode: str) : void
+destroy() : void
+resize() : void
contains data
«chart data»
ChartData
+labels: List[str]
+datasets: List[Dataset]
+add_data_point(value: float) : void
+remove_oldest_point() : void
contains datasets
«chart dataset»
Dataset
+label: str
+data: List[float]
+borderColor: str
+backgroundColor: str
+borderWidth: float
+pointRadius: float
+tension: float
+hidden: bool
+update_data(values: List[float]) : void
+toggle_visibility() : void